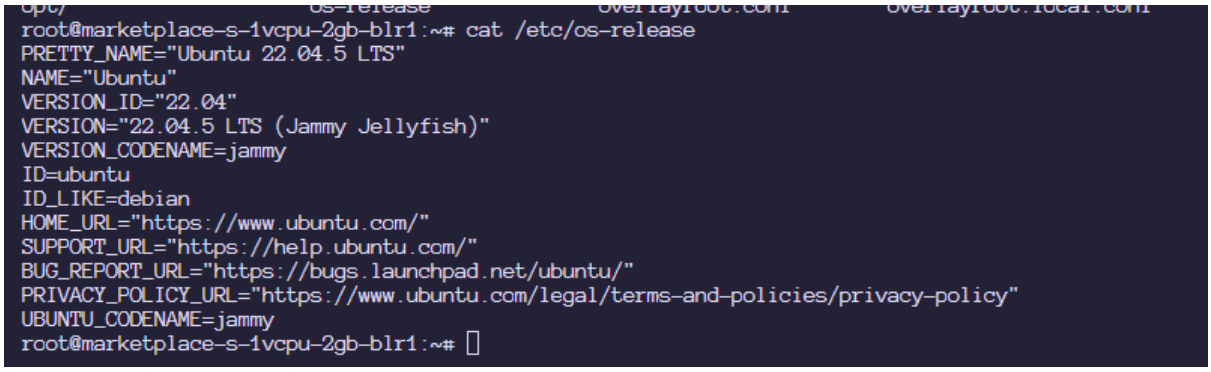
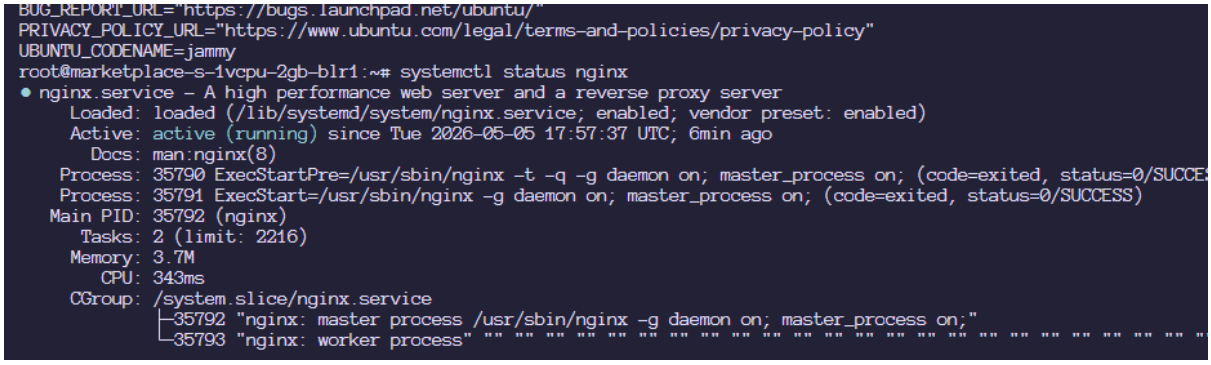
NAME: Subash Chandra Bose

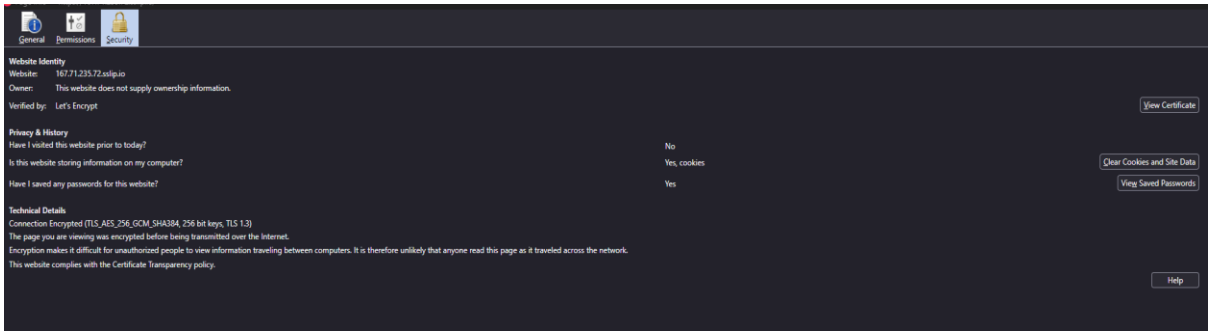
Set up Nginx on a VM with HTTPS. Test with SSL Labs. Iteratively fix issues (cipher suites, HSTS, OCSP) until you achieve A+.

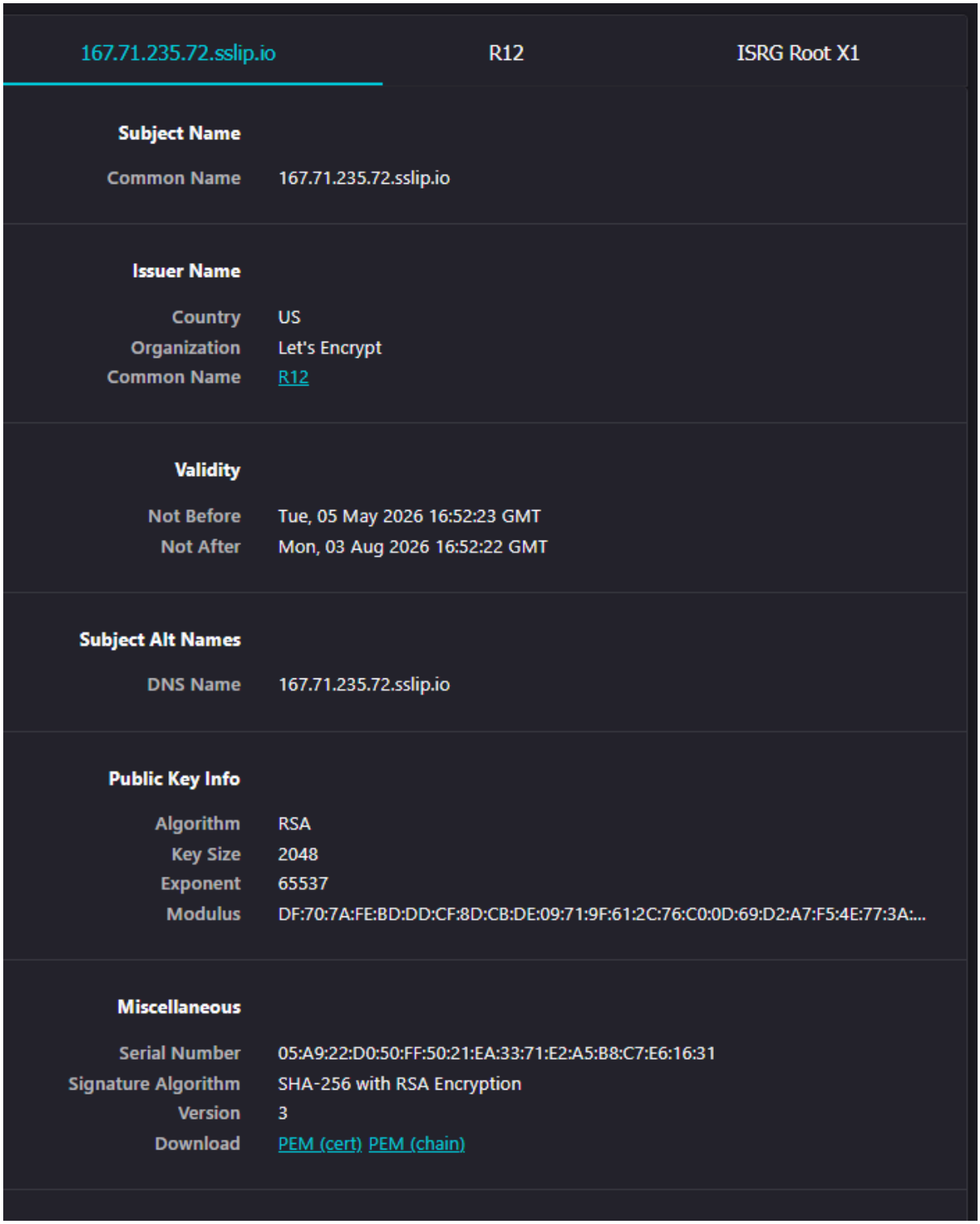




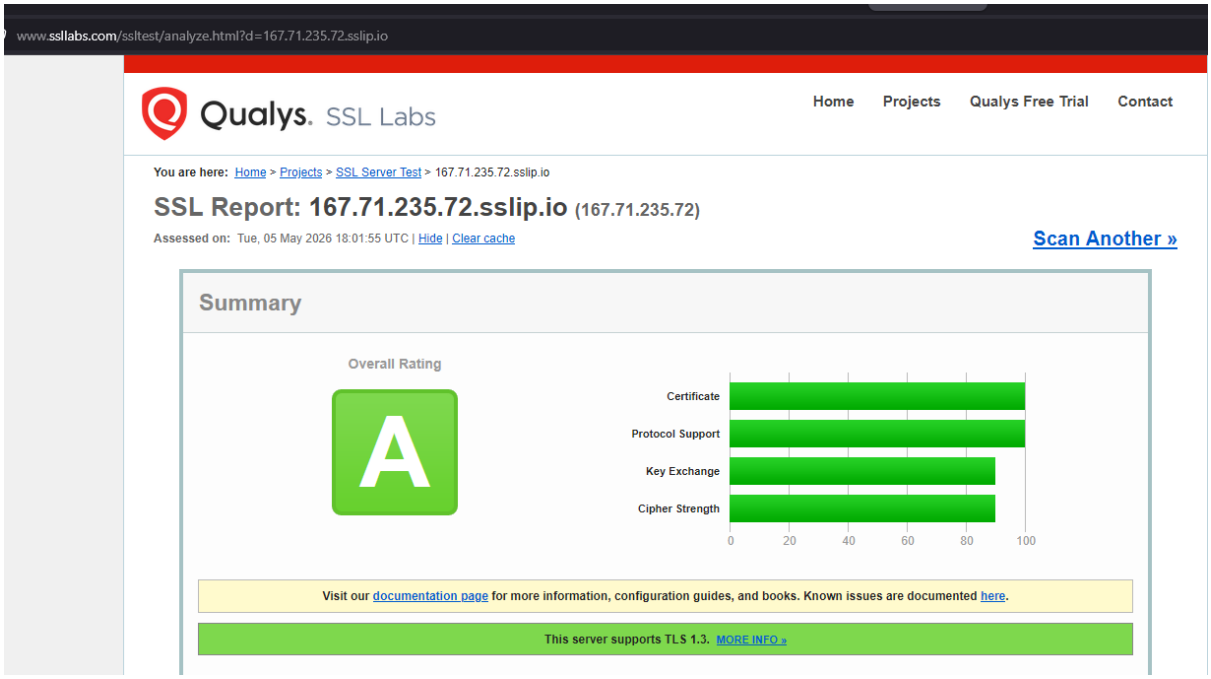
```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@marketplace-s-1vcpu-2gb-blr1:~# sudo certbot --nginx -d 167.71.235.72.sslip.io
Saving debug log to /var/log/letsencrypt/letsencrypt.log
Enter email address (used for urgent renewal and security notices)
 (Enter 'c' to cancel): bosemrgn@gmail.com

- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Please read the Terms of Service at
https://letsencrypt.org/documents/LE-SA-v1.6-August-18-2025.pdf. You must agree
in order to register with the ACME server. Do you agree?
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
(Y)es/(N)o: y

- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Would you be willing, once your first certificate is successfully issued, to
share your email address with the Electronic Frontier Foundation, a founding
partner of the Let's Encrypt project and the non-profit organization that
develops Certbot? We'd like to send you email about our work encrypting the web,
EFF news, campaigns, and ways to support digital freedom.
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
(Y)es/(N)o: y
Account registered.
Requesting a certificate for 167.71.235.72.sslip.io

Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/167.71.235.72.sslip.io/fullchain.pem
Key is saved at:         /etc/letsencrypt/live/167.71.235.72.sslip.io/privkey.pem
This certificate expires on 2026-08-03.
These files will be updated when the certificate renews.
```

```
  GNU nano 6.2
# HTTP server
server {
    listen 80;
    listen [::]:80;

    server_name 167.71.235.72.sslip.io;

    location / {
        root /var/www/html;
        index index.html;
    }
}

# HTTPS server
server {
    listen 443 ssl;
    listen [::]:443 ssl;

    server_name 167.71.235.72.sslip.io;

    root /var/www/html;
    index index.html;

    # Certificate (THIS WAS MISSING)
    ssl_certificate /etc/letsencrypt/live/167.71.235.72.sslip.io/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/167.71.235.72.sslip.io/privkey.pem;
}
```


General
Permissions
Security
Website Identity
Website: 167.71.235.72.sslip.io
Owner: This website does not supply ownership information.
Verified by: Let's Encrypt
View Certificate
Privacy & History
Have I visited this website prior to today? No
Is this website storing information on my computer? Yes, cookies
Clear Cookies and Site Data
Have I saved any passwords for this website? Yes
View Saved Passwords
Technical Details
Connection Encrypted (TLS_AES_256_GCM_SHA384, 256 bit keys, TLS 1.3)
The page you are viewing was encrypted before being transmitted over the Internet.
Encryption makes it difficult for unauthorized people to view information traveling between computers. It is therefore unlikely that anyone read this page as it traveled across the network.
This website complies with the Certificate Transparency policy.
Help

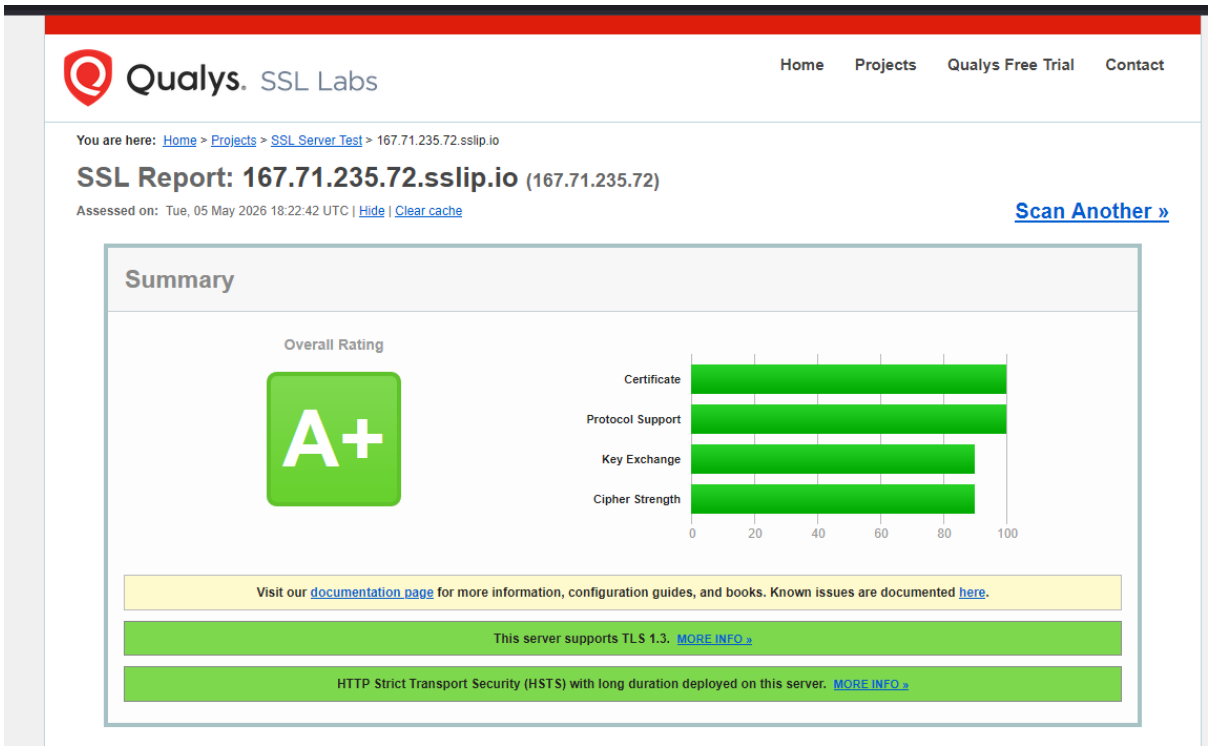

167.71.235.72.sslip.io
R12
ISRG Root X1
Subject Name
Common Name 167.71.235.72.sslip.io
Issuer Name
Country US
Organization Let's Encrypt
Common Name R12
Validity
Not Before Tue, 05 May 2026 16:52:23 GMT
Not After Mon, 03 Aug 2026 16:52:22 GMT
Subject Alt Names
DNS Name 167.71.235.72.sslip.io
Public Key Info
Algorithm RSA
Key Size 2048
Exponent 65537
Modulus DF:70:7A:FE:BD:DD:CF:8D:CB:DE:09:71:9F:61:2C:76:C0:0D:69:D2:A7:F5:4E:77:3A:...
Miscellaneous
Serial Number 05:A9:22:D0:50:FF:50:21:EA:33:71:E2:A5:B8:C7:E6:16:31
Signature Algorithm SHA-256 with RSA Encryption
Version 3
Download PEM (cert) PEM (chain)

Purposes	Server Authentication
Subject Key ID	
Key ID	1F:A1:7F:8B:33:B3:C0:84:45:ED:3D:69:E3:E7:1F:C3:C7:EF:4C:E8
Authority Key ID	
Key ID	00:B5:29:F2:2D:8E:6F:31:E8:9B:4C:AD:78:3E:FA:DC:E9:0C:D1:D2
CRL Endpoints	
Distribution Point	http://r12.c.lencr.org/50.crl
Authority Info (AIA)	
Location	http://r12.i.lencr.org/
Method	CA Issuers
Certificate Policies	
Policy	Certificate Type (2.23.140.1.2.1)
Value	Domain Validation
Embedded SCTs	
Log Name	Sectigo 'Tiger2026h2'
Log ID	C8:A3:C4:7F:C7:B3:AD:B9:35:6B:01:3F:6A:7A:12:6D:E3:3A:4E:43:A5:C6:46:F9:97:...
Signature Algorithm	SHA-256 ECDSA
Version	1
Timestamp	Tue, 05 May 2026 17:50:53 GMT
Log Name	IPng Networks 'Gouda2026h2'
Log ID	1A:8B:9D:6B:0F:FE:BF:81:B4:79:39:C6:D2:31:0A:86:D6:D1:02:D4:F0:46:E2:18:2C:9...
Signature Algorithm	SHA-256 ECDSA
Version	1
Timestamp	Tue, 05 May 2026 17:50:54 GMT



For A+ I have changed:

```
root@marketplace-s-1vcpu-2gb-blr1:~# sudo systemctl restart nginx
root@marketplace-s-1vcpu-2gb-blr1:~# sudo cat /etc/nginx/sites-available/default
# ------------------------------
# HTTP → HTTPS Redirect
# ------------------------------
server {
    listen 80;
    listen [::]:80;

    server_name 167.71.235.72.sslip.io;

    return 301 https://$host$request_uri;
}

# ------------------------------
# HTTPS Server (A+ Config)
# ------------------------------
server {
    listen 443 ssl http2;
    listen [::]:443 ssl http2;

    server_name 167.71.235.72.sslip.io;

    root /var/www/html;
    index index.html index.htm;

    # 🔒 Let's Encrypt Certificate
    ssl_certificate /etc/letsencrypt/live/167.71.235.72.sslip.io/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/167.71.235.72.sslip.io/privkey.pem;

    # 🔒 TLS Protocols
    ssl_protocols TLSv1.2 TLSv1.3;

    # 🔒 Strong Ciphers
    ssl_prefer_server_ciphers on;
    ssl_ciphers EECDH+AESGCM:EDH+AESGCM;

    # 🔒 Session Settings
    ssl_session_cache shared:SSL:10m;
    ssl_session_timeout 1d;
    ssl_session_tickets off;

    # 🔒 HSTS (needed for A+)
    add_header Strict-Transport-Security "max-age=63072000; includeSubDomains; preload" always;

    # 🔒 OCSP Stapling (fixed)
    ssl_stapling on;
    ssl_stapling_verify on;

    # 🔒 Resolver (fixes warning)
    resolver 8.8.8.8 1.1.1.1 valid=300s;
    resolver_timeout 5s;

    # ------------------------------
    # Website content
    # ------------------------------
    location / {
        try_files $uri $uri/ =404;
    }
}
```


Qualys. SSL Labs
Home
Projects
Qualys Free Trial
Contact
You are here: Home > Projects > SSL Server Test > 167.71.235.72.sslip.io
SSL Report: 167.71.235.72.sslip.io (167.71.235.72)
Assessed on: Tue, 05 May 2026 18:22:42 UTC | Hide | Clear cache
Scan Another »
Summary
Overall Rating
A+
Certificate
Protocol Support
Key Exchange
Cipher Strength
0
20
40
60
80
100
Visit our documentation page for more information, configuration guides, and books. Known issues are documented here.
This server supports TLS 1.3. MORE INFO »
HTTP Strict Transport Security (HSTS) with long duration deployed on this server. MORE INFO »

POODLE (SSLv3)	No, SSL 3 not supported (more info)
POODLE (TLS)	No (more info)
Zombie POODLE	No (more info)
GOLDENDOODLE	No (more info)
OpenSSL 0-Length	No (more info)
Sleeping POODLE	No (more info)
Downgrade attack prevention	**Yes, TLS_FALLBACK_SCSV supported** (more info)
SSL/TLS compression	No
RC4	No
Heartbeat (extension)	No
Heartbleed (vulnerability)	No (more info)
Ticketbleed (vulnerability)	No (more info)
OpenSSL CCS vuln. (CVE-2014-0224)	No (more info)
OpenSSL Padding Oracle vuln. (CVE-2016-2107)	No (more info)
ROBOT (vulnerability)	No (more info)
Forward Secrecy	**Yes (with most browsers) ROBUST** (more info)
ALPN	Yes h2 http/1.1
NPN	Yes h2 http/1.1
Session resumption (caching)	Yes
Session resumption (tickets)	No
OCSP stapling	No
Strict Transport Security (HSTS)	**Yes** max-age=63072000; includeSubDomains; preload
HSTS Preloading	Not in: Chrome Edge Firefox IE
Public Key Pinning (HPKP)	No (more info)
Public Key Pinning Report-Only	No
Public Key Pinning (Static)	No (more info)
Long handshake intolerance	No
TLS extension intolerance	No
TLS version intolerance	No
Incorrect SNI alerts	No
Uses common DH primes	No, DHE suites not supported
DH public server param (Ys) reuse	No, DHE suites not supported
ECDH public server param reuse	No

```
^C
root@marketplace-s-1vcpu-2gb-blr1:~# openssl s_client -connect 167.71.235.72.sslip.io:443 -servername 167.71.235.72
CONNECTED(00000003)
depth=2 C = US, O = Internet Security Research Group, CN = ISRG Root X1
verify return:1
depth=1 C = US, O = Let's Encrypt, CN = R12
verify return:1
depth=0 CN = 167.71.235.72.sslip.io
verify return:1
---
Certificate chain
 0 s:CN = 167.71.235.72.sslip.io
   i:C = US, O = Let's Encrypt, CN = R12
   a:PKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
   v:NotBefore: May  5 16:52:23 2026 GMT; NotAfter: Aug  3 16:52:22 2026 GMT
 1 s:C = US, O = Let's Encrypt, CN = R12
   i:C = US, O = Internet Security Research Group, CN = ISRG Root X1
   a:PKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
   v:NotBefore: Mar 13 00:00:00 2024 GMT; NotAfter: Mar 12 23:59:59 2027 GMT
---
Server certificate
-----BEGIN CERTIFICATE-----
MIIFBjCCA+6gAwIBAgISBaki0FD/UCHqM3HipbjH5hYxMA0GCSqGSIb3DQEBCwUA
MDMxCzAJBgNVBAYTAlVTMRYwFAYDVQQKEw1MZXQncyBFbmNyeXB0MQwwCgYDVQQD
EwNSMTIwHhcNMjYwNTA1MTY1MjIzWhcNMjYwODAzMTY1MjIyWjAhMR8wHQYDVQQD
ExYxNjcuNzEuMjM1LjcyLnNzbGlwLmlvMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8A
MIIBCgKCAQEA33B6/r3dz43L3glxn2EsdsANadKn9U53OuhUTH1KufKxSn6O9W+B
QSiXp16cLP7O95NeJis9MZyE+O/4rrRe/eleEpRgAu72uaiqDu5X8EgltAAawiOw
PfH00aRpFGbGQJsvfm3TQlQ2MmGLjKJsHSF4e9K0MG0fw4fCnlI94eQCKDEEhql
Nof7rviTBoSSvh+HJb8yaaJFCUT0NXZ3aU3dJ/K1mnQ7ycgbJBev5cDkkiASHCBF
QILPKdGcQtAai+VXlTnJs12OgxIdj2VoeUEFIWHPV2GpCJSOCWWKPqpCwLnSVhAl
Om1CnbnvVJIXdTF4k9rDlnkbgtXJ3PU3lwIDAQABo4ICJDCCAiAwDgYDVR0PAQH/
BAQDAgWgMBMGA1UdJQQMMAoGCCsGAQUFBwMBMAwGA1UdEwEB/wQCMAAwHQYDVR0O
BBYEFB+hf4szs8CERe09aePnH8PH70zoMB8GA1UdIwQYMBaAFAC1KfItjm8x6JtM
rXg++tzpDNHSMDMGCCsGAQUFBwEBBCcwJTAjBggrBgEFBQcwAoYXaHR0cDovL3Ix
Mi5pLmxlbmNyLm9yZy8wIQYDVR0RBBowGIIWMTY3LjcxLjIzNS43Mi5zc2xpcC5p
bzATBgNVHSAEDDAKMAgGBmeBDAECATAuBgNVHR8EJzAlMCOgIaAfhh1odHRwOi8v
cjEyLmMubGVuY3Iub3JnLzUwLmNybDCCAQwGCisGAQQB1nkCBAIEgf0EgfoA+AB2
AMijxH/Hs625NWsBP2p6Em3jOk5DpcZG+ZetOXWZHc+aAAABnflDp9QAAAQDAEcw
RQIgQYA6gJr9iqNtL2d4yElNwik29XKs28I45zILxMvbWEECIQDRw/ZHek/SRtiw
IYs5SWjDRp/7s87Zo6Q/dU0TSmusPwB+ABqLnWsP/r+BtHk5xtIxCobW0QLU8Ebi
GCyd419eJiXvAAABnflDqu0ACAAABQAPSdhJBAMARzBFAiEAr/JMTVuY5CCFH0dW
PtBqvcSacL/zm6RPMEOUf2N1mwECIHhGA8x0Oegm5yotUvavCmrb5+89OI6hrsXj
B6LTmqWuMA0GCSqGSIb3DQEBCwUAA4IBAQCfrdJ9Zn4ZU7M2XESTAJUhRhLnfj7R
sYGcQn8vrpERUEBA935qpfn3EmSzT+8eVcrh2rsviNvjbXIp64pL+/78106hVZyr
YQqqR5e64REtRzsAJluTzV1bIvU6pcVFDUUmlvnufyUHsX30CwO5SLaKWNZA4nWS
oZHJB+7oHaffmVmECDcUY1Czfq9KfeK3CK035NPDcZrOQU3SXqvRISmKiHr76LCK
mGyTpY+63G0CkXS4KIoKkycEvFAxRe2NZY6+Fv3Rx1LUGxxiydwmX0SNq2Pux2zt
OdvBZ0ZrQRzTckYAhRAXchohq2S7qEHXtL8Mko/xSNohL1HfUCHc7+7c
-----END CERTIFICATE-----
subject=CN = 167.71.235.72.sslip.io
issuer=C = US, O = Let's Encrypt, CN = R12
---
No client certificate CA names sent
Peer signing digest: SHA256
Peer signature type: RSA-PSS
Server Temp Key: X25519, 253 bits
---
SSL handshake has read 3145 bytes and written 404 bytes
Verification: OK
---
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Server public key is 2048 bit
Secure Renegotiation IS NOT supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)
---
```

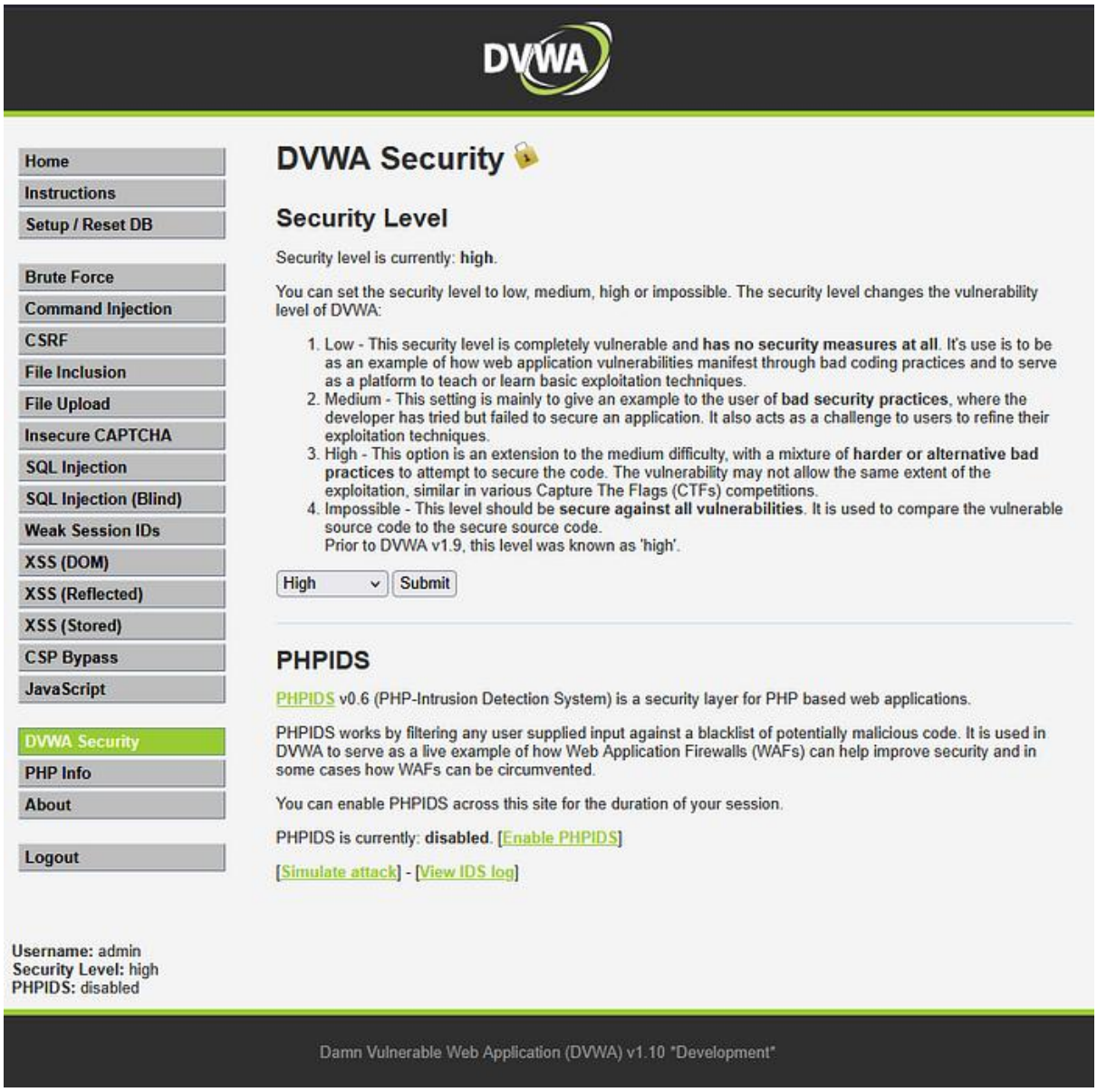
Conclusion

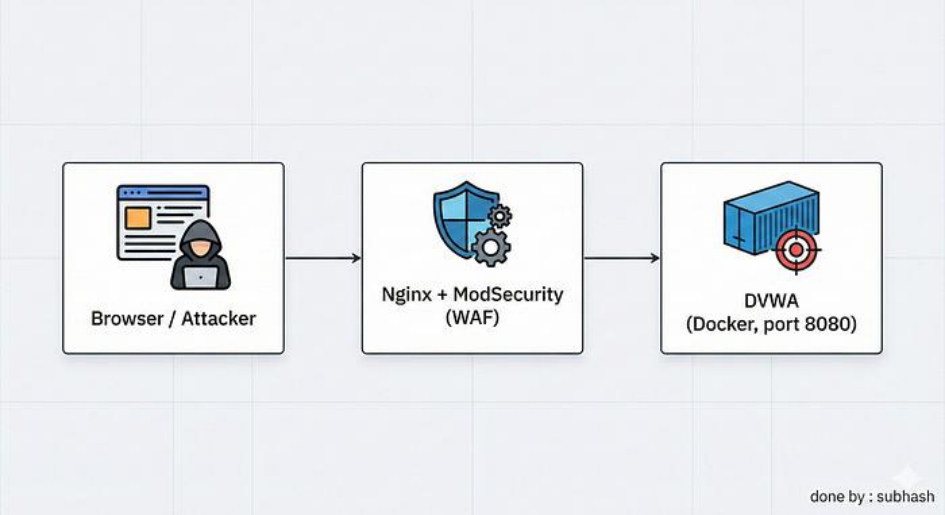
In this experiment, a secure web server was successfully configured using Nginx on a cloud-based Ubuntu system. A trusted SSL/TLS certificate was obtained using Let's Encrypt and integrated into the server to enable HTTPS communication. Initially, the server operated with default configurations, resulting in a lower SSL rating. By applying strong security measures such as enforcing TLS 1.2 and TLS 1.3 protocols, configuring secure cipher suites, enabling HTTP to HTTPS redirection, and implementing HTTP Strict Transport Security (HSTS), the overall security posture of the server was significantly improved.

The final configuration was validated using SSL Labs, where the server achieved an A+ rating, demonstrating compliance with modern web security standards. This experiment provided practical understanding of SSL/TLS implementation, certificate management, and secure server configuration, which are essential for building and maintaining secure web applications.

---

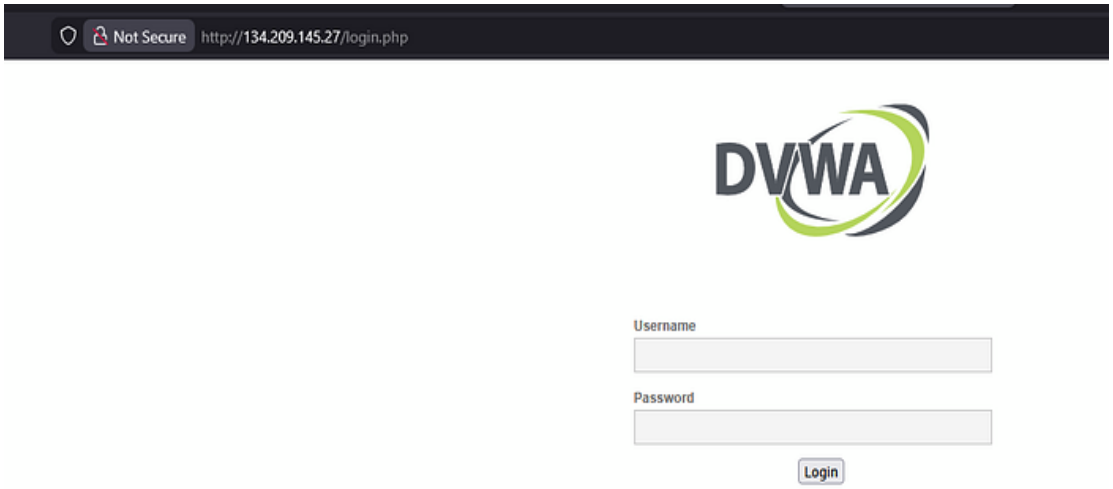
2) Install DVWA or Juice Shop. Place ModSecurity WAF in front of it with OWASP Core Rule Set. Verify it blocks SQLi and XSS.

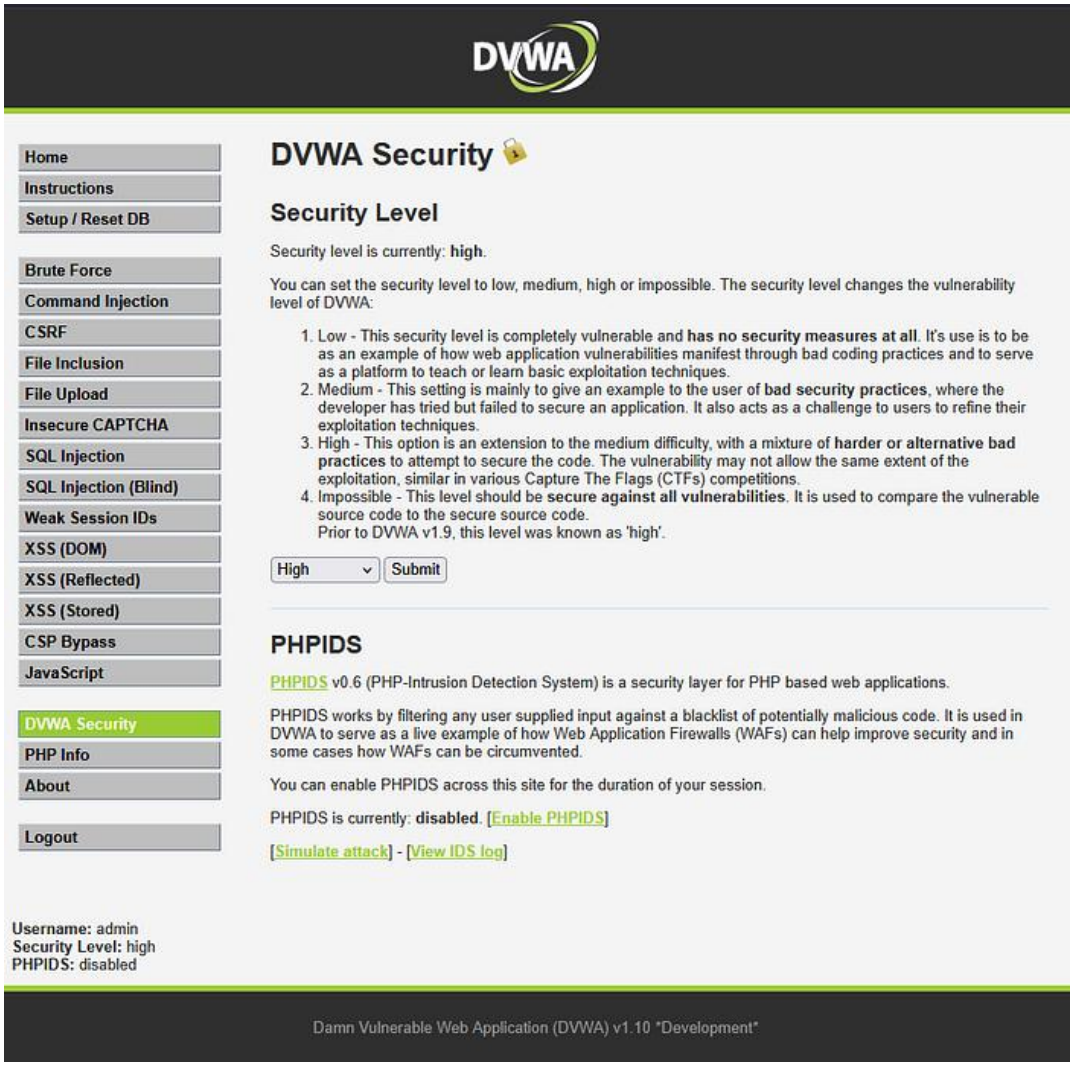





```
subhash@ubuntu-s-2vcpu-8gb-160gb-intel-blr1-01:/etc/nginx$ docker ps -a
CONTAINER ID   IMAGE                  COMMAND                  CREATED       STATUS       PORTS                                   NAMES
0aff42b7d66d   vulnerables/web-dvwa   "/main.sh"               4 hours ago   Up 4 hours   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   dvwa-security-lab_dvwa_1
c9b398241895   mysql:5.7              "docker-entrypoint.s…"   4 hours ago   Up 4 hours   3306/tcp, 33060/tcp                       dvwa-security-lab_db_1
subhash@ubuntu-s-2vcpu-8gb-160gb-intel-blr1-01:/etc/nginx$
[0] 0:bash*
```

```
subhash@ubuntu-s-2vcpu-8gb-160gb-intel-blr1-01:~/dvwa-security-lab$ cat docker-comp
version: '3'

services:
  dvwa:
    image: vulnerables/web-dvwa
    ports:
      - "8080:80"
    environment:
      MYSQL_HOST: db
      MYSQL_DATABASE: dvwa
      MYSQL_USER: dvwa
      MYSQL_PASSWORD: dvwa
    depends_on:
      - db

  db:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: dvwa
      MYSQL_USER: dvwa
      MYSQL_PASSWORD: dvwa
subhash@ubuntu-s-2vcpu-8gb-160gb-intel-blr1-01:~/dvwa-security-lab$
[0] 0:bash*
```





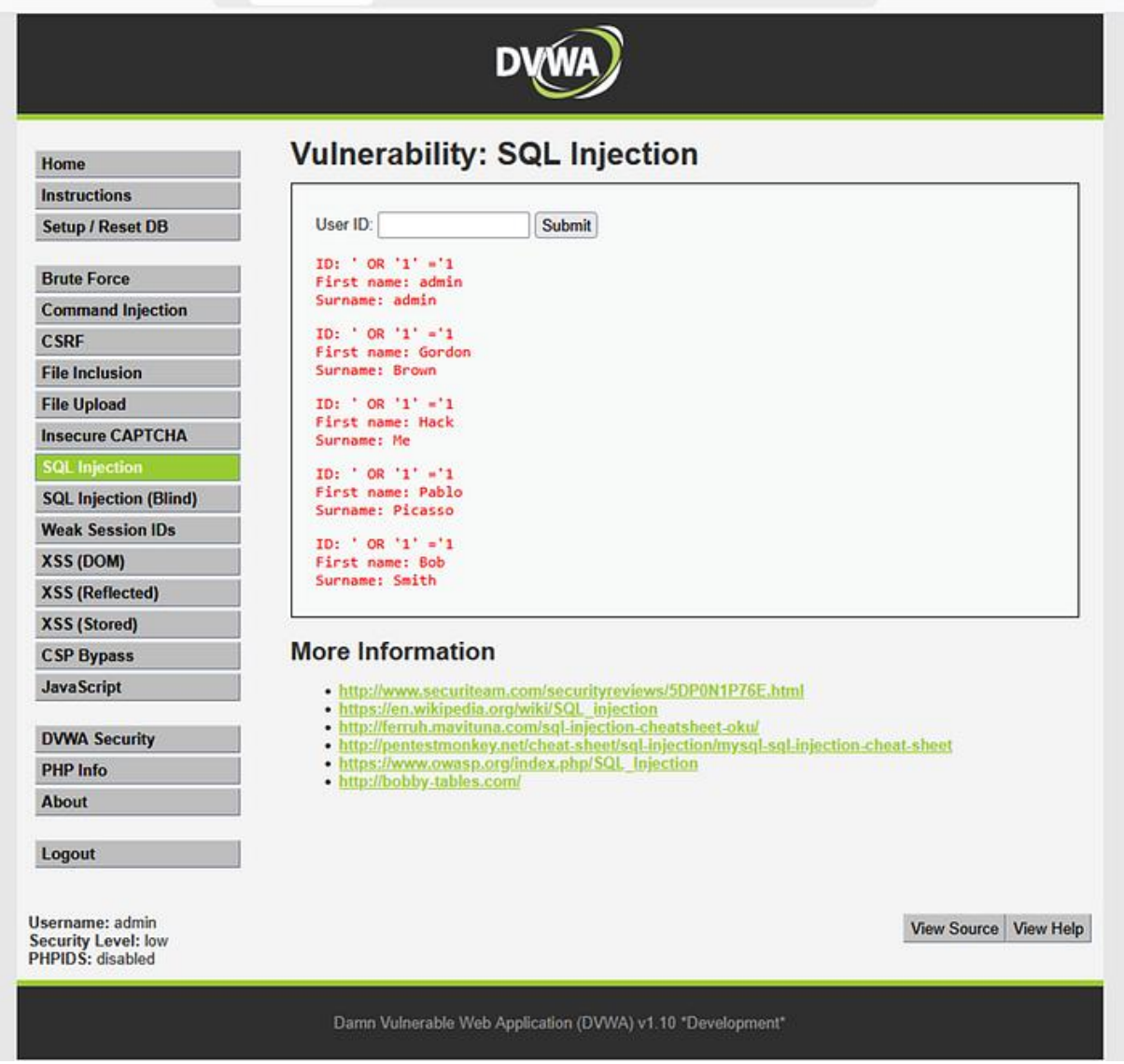
Step 1 : Attacking DVWA without a WAF (Reality Check)

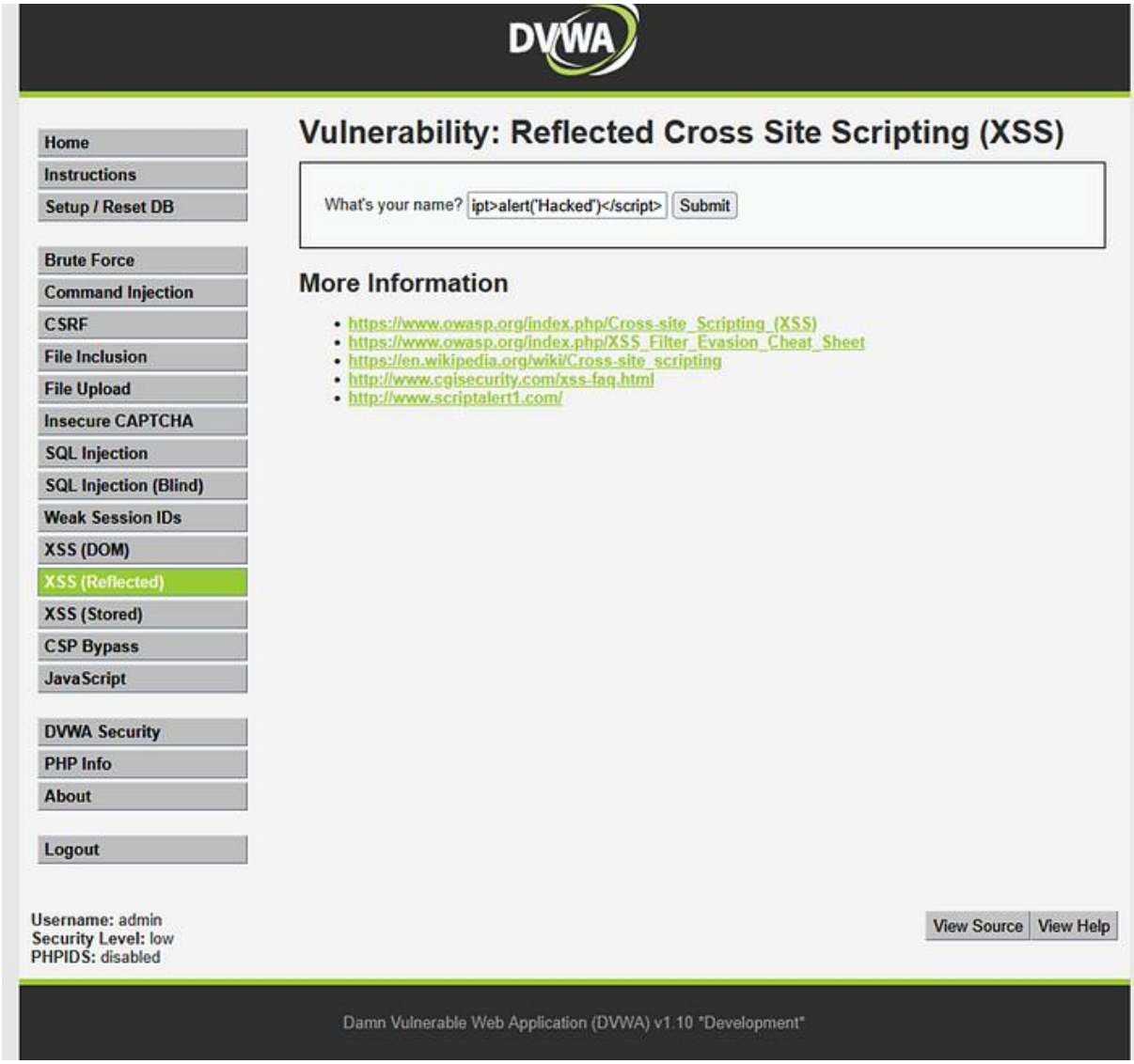
Before even touching ModSecurity, I wanted to see **how bad things actually are** when an app runs *without* a Web Application Firewall.

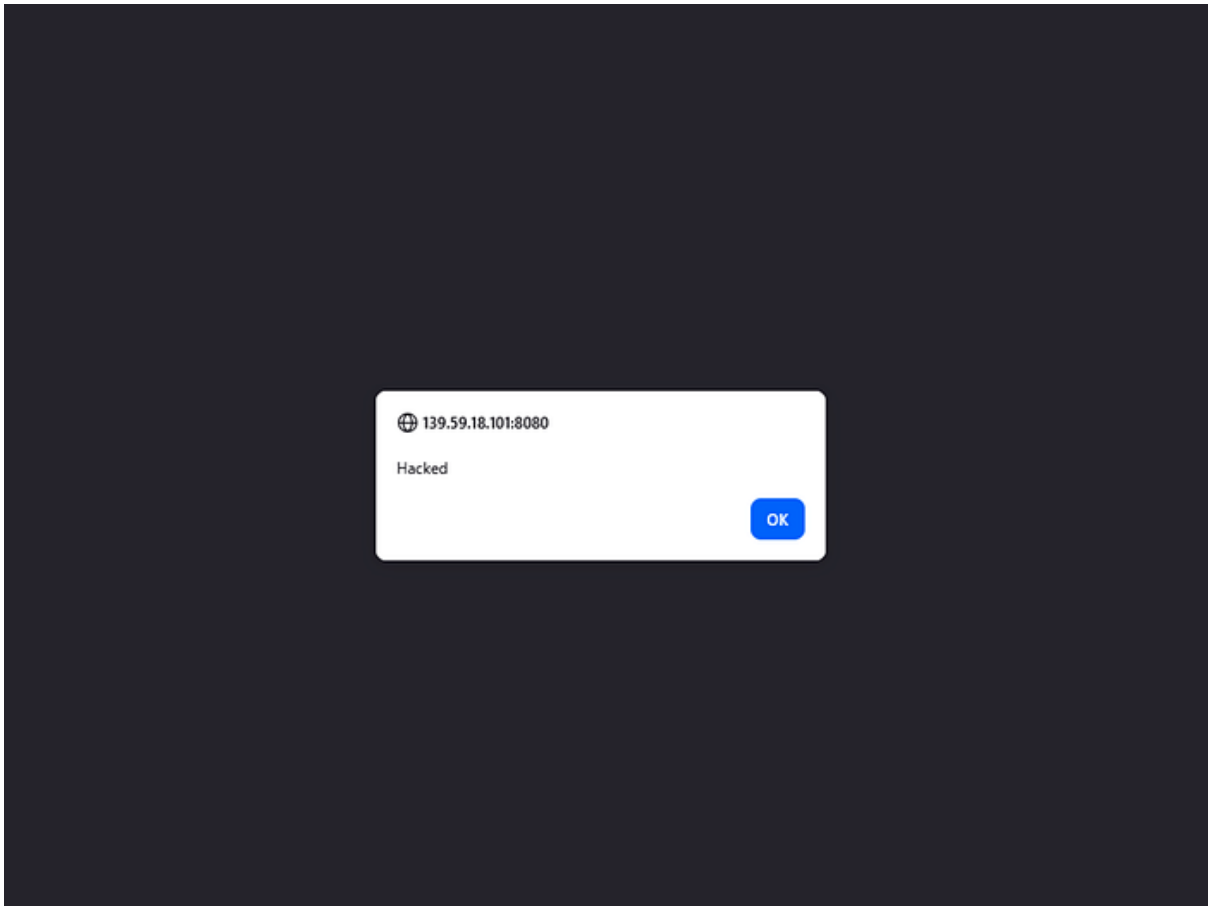
So I deployed **DVWA directly on Docker (port 8080)** and tried a few classic attacks.

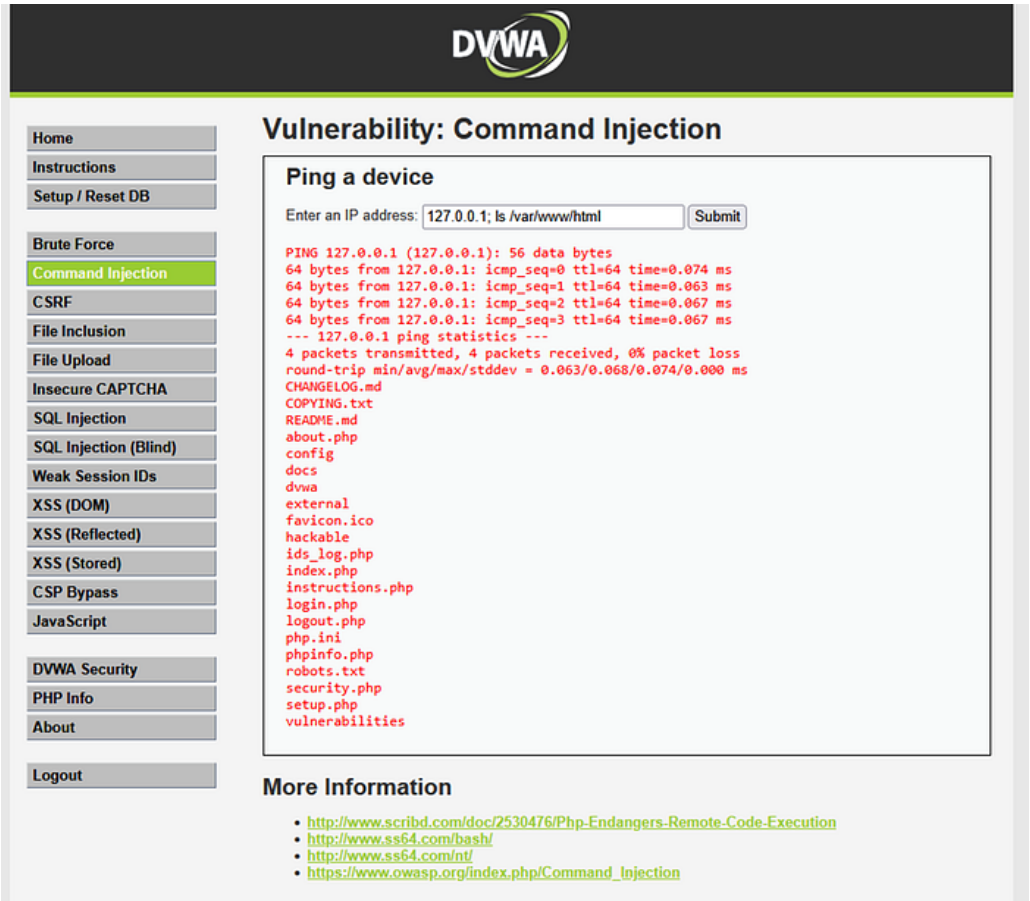
What I tested:

- SQL Injection
- Command Injection
- XSS (Reflected & Stored)




DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript
DVWA Security
PHP Info
About
Logout
Vulnerability: Reflected Cross Site Scripting (XSS)
What's your name? ipt>alert('Hacked')</script> Submit
More Information
https://www.owasp.org/index.php/Cross-site_Scripting_(XSS)
https://www.owasp.org/index.php/XSS_Filter_Evasion_Cheat_Sheet
https://en.wikipedia.org/wiki/Cross-site_scripting
http://www.cgisecurity.com/xss-faq.html
http://www.scriptalert1.com/
Username: admin
Security Level: low
PHPIDS: disabled
View Source
View Help
Damn Vulnerable Web Application (DVWA) v1.10 *Development*


139.59.18.101:8080
Hacked
OK



This step was important because:

- I needed a **baseline**
- Otherwise there's no proof that ModSecurity actually *did* something later

Once I confirmed attacks were working freely, **then only** I moved to installing a WAF.

Installing ModSecurity with Nginx.

Initially, ModSecurity was enabled globally inside the http {} block.

However, this applies WAF rules to every hosted application, which is not ideal.

To follow a more realistic and controlled approach, I enabled ModSecurity only at the virtual host level for DVWA.

This allows targeted protection, cleaner debugging, and mirrors real-world WAF deployments.

Why?

- Cleaner setup
- Real-world approach
- No unnecessary blocking elsewhere

```
subhash@ubuntu-s-2vcpu-8gb-160gb-intel-blr1-01:/etc/nginx/sites-enabled$ ls
dvwa
subhash@ubuntu-s-2vcpu-8gb-160gb-intel-blr1-01:/etc/nginx/sites-enabled$ cat dvwa
server {
    listen 80;
    server_name _;

    # Enable ModSecurity
    modsecurity on;
    modsecurity_rules_file /etc/nginx/modsecurity.conf;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
subhash@ubuntu-s-2vcpu-8gb-160gb-intel-blr1-01:/etc/nginx/sites-enabled$
[0] 0:bash*
```

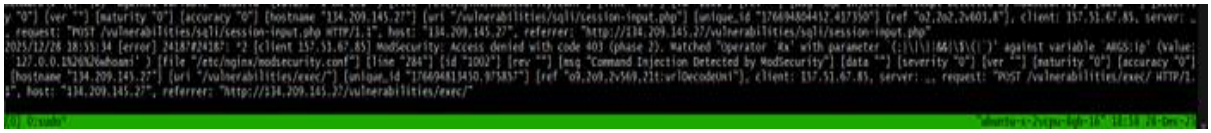
Now the waf is infront of DVWA.

Writing Custom WAF Rules

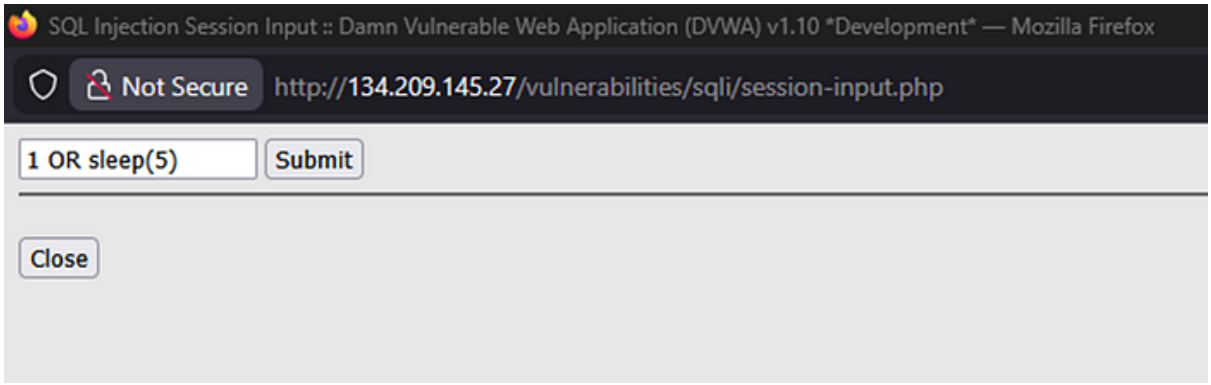
Instead of using CRS directly, I wrote **minimal custom rules** to clearly prove blocking behavior.

```
# ================================================
# Custom minimal WAF rules (DVWA protection)
# ================================================

# Command Injection

#SecRule ARGS "@rx (;|\|\||&&|\$\(|`)" \
#"id:1002,phase:2,t:urlDecodeUni,log,deny,status:403,msg:'Command Injection Detected by ModSecurity'"
SecRule ARGS "@rx (;|\|\||&&|\$\(|`)" \
"id:1002,phase:2,t:urlDecodeUni,log,deny,status:403,msg:'Command Injection Detected by ModSecurity'"


# Sql Injection
SecRule ARGS "@rx (?i)(\bunion\b|\bselect\b|\bor\b|\band\b|sleep\s*\(|benchmark\s*\(|--|#)" \
"id:1001,phase:2,log,deny,status:403,msg:'SQL Injection Attempt Detected by ModSecurity'"


# XSS Injection
SecRule ARGS "@rx (?i)(<script|</script|onerror=|onload=|alert\s*\(|document\.cookie|<img)" \
"id:1003,phase:2,deny,status:403,msg:'XSS Attack Detected'"
subhash@ubuntu-s-2vcpu-8gb-160gb-intel-blr1-01:/etc/nginx$
[0] 0:bash*
```

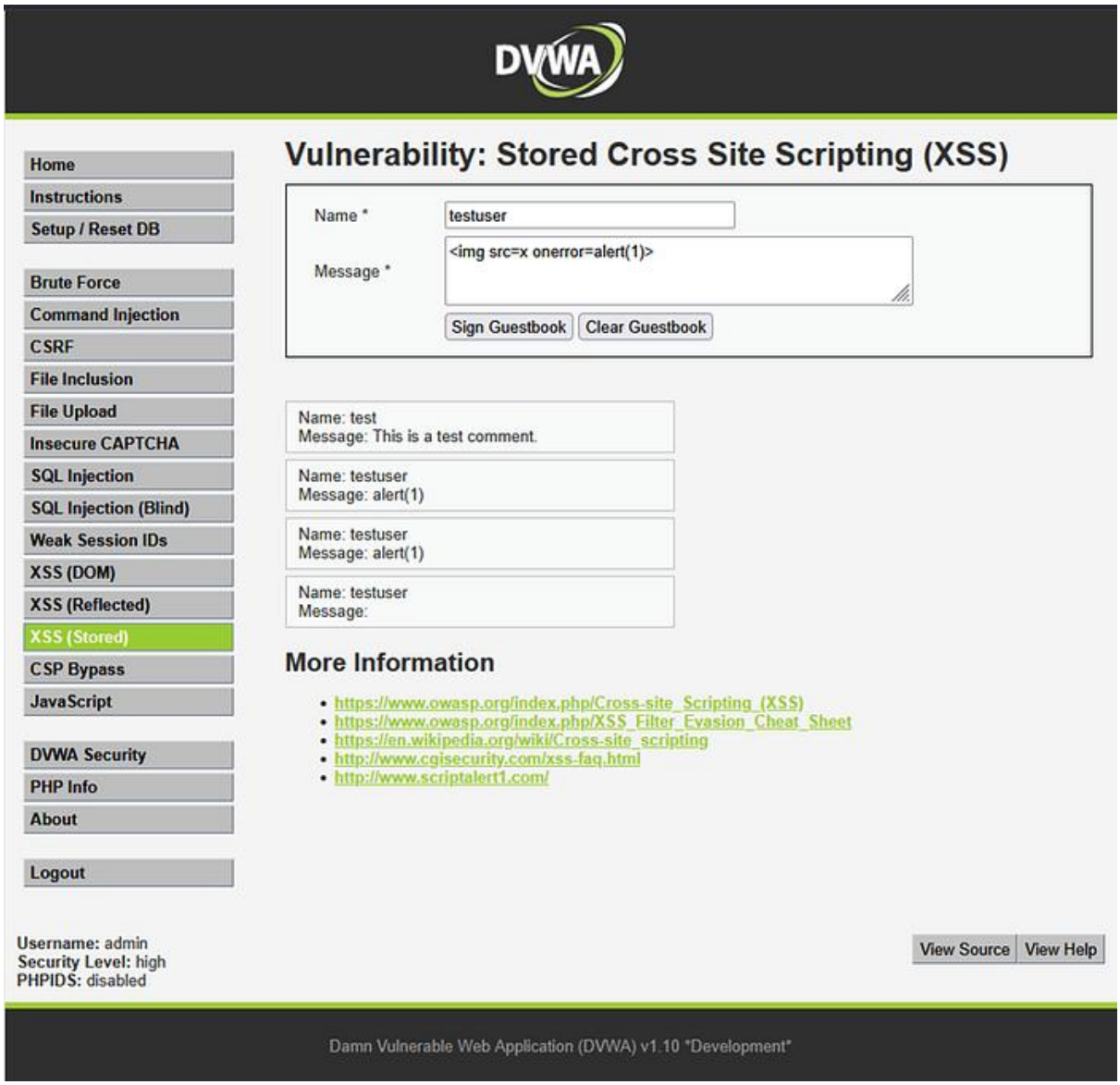
Attacking the DVWA:



SQL Injection:



XSS Attempt:


DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript
DVWA Security
PHP Info
About
Logout
Vulnerability: Stored Cross Site Scripting (XSS)
Name *
testuser
Message *
<img src=x onerror=alert(1)>
Sign Guestbook
Clear Guestbook
Name: test
Message: This is a test comment.
Name: testuser
Message: alert(1)
Name: testuser
Message: alert(1)
Name: testuser
Message:
More Information
https://www.owasp.org/index.php/Cross-site_Scripting_(XSS)
https://www.owasp.org/index.php/XSS_Filter_Evasion_Cheat_Sheet
https://en.wikipedia.org/wiki/Cross-site_scripting
http://www.cgisecurity.com/xss-faq.html
http://www.scriptalert1.com/
Username: admin
Security Level: high
PHPIDS: disabled
View Source
View Help
Damn Vulnerable Web Application (DVWA) v1.10 "Development"

Conclusion:

When ModSecurity was introduced as a Web Application Firewall in front of DVWA, the story changed. The same attacks that worked earlier were now **blocked**, **logged**, and **controlled**. Seeing 403 responses instead of successful exploitation made it clear how powerful a properly configured WAF can be.